



Universidade Federal do Ceará
Campus Quixadá

QXD0011 - Fundamentos de Banco de Dados
Prof. Francisco Victor da Silva Pinheiro

Trabalho Final

Entrega 3 – Modelo Relacional e Implementação no SGBD

Quixadá-CE
2026

1. Integrantes da Equipe

Matrícula	Nome	E-mail
554930	Francisca Ariane dos Santos da Silva	francisca.silva@alu.ufc.br
493659	Gabriel Braga Martins	gabrielbragamartins@alu.ufc.br
587754	Kailany Sofia Alves de Lima	kailanysofia@alu.ufc.br
539730	Pablo Brandão Passos	pablobrandao@alu.ufc.br
596042	Raul Camurça Rabelo de Almeida	raulcamurca@alu.ufc.br

2. Resumo da Proposta

Nesta etapa do projeto UniCarona, foi realizado o mapeamento do Modelo ER para o modelo relacional, além da preparação para implementação do banco de dados em um SGBD. O modelo foi revisado e ajustado com base no feedback da entrega anterior, buscando maior adequação aos conceitos de modelagem conceitual e relacional.

Foram definidas as tabelas, chaves primárias e chaves estrangeiras necessárias para representar entidades como usuários, motoristas, veículos, caronas, reservas, mensagens, avaliações, pagamentos e locais. O objetivo desta etapa é garantir uma estrutura de banco de dados organizada, normalizada e coerente com as funcionalidades propostas pelo sistema UniCarona.

3. Mapeamento ER → Relacional

O modelo relacional abaixo foi elaborado a partir do DER revisado da etapa anterior, contendo as tabelas, atributos, chaves primárias e chaves estrangeiras necessárias para implementação do banco de dados do sistema UniCarona. Tanto em notação textual quanto em diagrama.

USUARIO(
cpf PK,
nome,
email,
telefone
)

MOTORISTA(
cpf PK FK → USUARIO(cpf),
numero_cnh,
validade_cnh
)

VEICULO(
placa PK,
cpf_motorista FK → MOTORISTA(cpf),
modelo,
marca,
cor,
capacidade_passageiros
)

LOCAL(
id_local PK,
nome_local,
cidade,
bairro
)

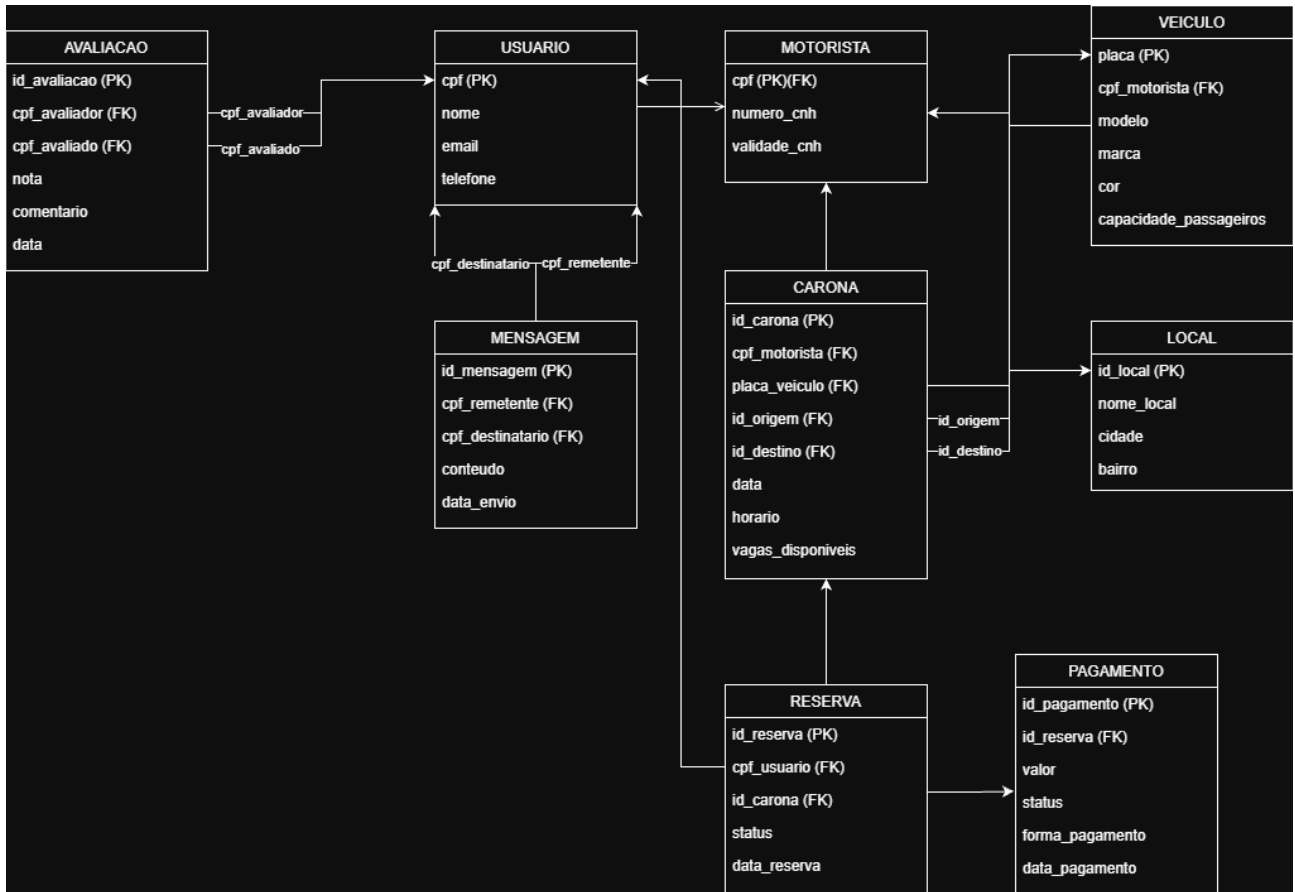
CARONA(
id_carona PK,
cpf_motorista FK → MOTORISTA(cpf),
placa_veiculo FK → VEICULO(placa),
id_origem FK → LOCAL(id_local),
id_destino FK → LOCAL(id_local),
data,
horario,
vagas_disponiveis
)

RESERVA(
id_reserva PK,
cpf_usuario FK → USUARIO(cpf),
id_carona FK → CARONA(id_carona),
status,
data_reserva
)

MENSAGEM(
id_mensagem PK,
cpf_remetente FK → USUARIO(cpf),
cpf_destinatario FK → USUARIO(cpf),
conteudo,
data_envio
)

AVALIACAO(
id_avaliacao PK,
cpf_avalador FK → USUARIO(cpf),
cpf_avalado FK → USUARIO(cpf),
nota,
comentario,
data
)

PAGAMENTO(
 id_pagamento PK,
 id_reserva FK → RESERVA(id_reserva),
 valor,
 status,
 forma_pagamento,
 data_pagamento
)



-Também disponível em [UniCarona diagrama relacional.drawio](#)

4. Implementação no SGBD (SQL)

- **Criação do esquema (DDL)**

```

-- =====
-- PARTE 1: COMANDOS DDL (CRIAÇÃO DAS ESTRUTURAS)
-- =====

DROP TABLE IF EXISTS PAGAMENTO CASCADE;

DROP TABLE IF EXISTS AVALIACAO CASCADE;

DROP TABLE IF EXISTS MENSAGEM CASCADE;

```

```

DROP TABLE IF EXISTS RESERVA CASCADE;

DROP TABLE IF EXISTS CARONA CASCADE;

DROP TABLE IF EXISTS LOCAL CASCADE;

DROP TABLE IF EXISTS VEICULO CASCADE;

DROP TABLE IF EXISTS MOTORISTA CASCADE;

DROP TABLE IF EXISTS USUARIO CASCADE;


-- CRIANDO TABELA USUARIO

CREATE TABLE USUARIO (

    cpf VARCHAR(11) NOT NULL,

    nome VARCHAR(100) NOT NULL,

    email VARCHAR(100) NOT NULL UNIQUE,

    telefone VARCHAR(15),

    CONSTRAINT PK_USUARIO PRIMARY KEY (cpf)

);


-- CRIANDO TABELA MOTORISTA

CREATE TABLE MOTORISTA (

    cpf VARCHAR(11) NOT NULL,

    numero_cnh VARCHAR(20) NOT NULL UNIQUE,

    validade_cnh DATE NOT NULL,

    CONSTRAINT PK_MOTORISTA PRIMARY KEY (cpf),

    CONSTRAINT FK_MOTORISTA_USUARIO FOREIGN KEY (cpf) REFERENCES
USUARIO(cpf) ON DELETE CASCADE

);

```

```

-- CRIANDO TABELA VEICULO

CREATE TABLE VEICULO (

    placa VARCHAR(7) NOT NULL,

    cpf_motorista VARCHAR(11) NOT NULL,

    modelo VARCHAR(50) NOT NULL,

    marca VARCHAR(50) NOT NULL,

    cor VARCHAR(30),

    capacidade_passageiros INT NOT NULL,

    CONSTRAINT PK_VEICULO PRIMARY KEY (placa),

    CONSTRAINT FK_VEICULO_MOTORISTA FOREIGN KEY (cpf_motorista)
REFERENCES MOTORISTA(cpf) ON DELETE CASCADE,

    CONSTRAINT UNQ_VEICULO_MOTORISTA UNIQUE (placa, cpf_motorista)

);

-- CRIANDO TABELA LOCAL

CREATE TABLE LOCAL (

    id_local INT NOT NULL,

    nome_local VARCHAR(100) NOT NULL,

    cidade VARCHAR(50) NOT NULL,

    bairro VARCHAR(50),

    CONSTRAINT PK_LOCAL PRIMARY KEY (id_local)

);

-- CRIANDO TABELA CARONA

CREATE TABLE CARONA (

    id_carona INT NOT NULL,

```

```

    cpf_motorista VARCHAR(11) NOT NULL,

    placa_veiculo VARCHAR(7) NOT NULL,

    id_origem INT NOT NULL,

    id_destino INT NOT NULL,

    data DATE NOT NULL,

    horario TIME NOT NULL,

    vagas_disponiveis INT NOT NULL,

    CONSTRAINT PK_CARONA PRIMARY KEY (id_carona),

    CONSTRAINT FK_CARONA_VEICULO FOREIGN KEY (placa_veiculo) REFERENCES
VEICULO(placa),

    CONSTRAINT FK_CARONA_MOTORISTA FOREIGN KEY (cpf_motorista)
REFERENCES MOTORISTA(cpf),

    CONSTRAINT FK_CARONA_ORIGEM FOREIGN KEY (id_origem) REFERENCES
LOCAL(id_local),

    CONSTRAINT FK_CARONA_DESTINO FOREIGN KEY (id_destino) REFERENCES
LOCAL(id_local)
);

-- CRIANDO TABELA RESERVA

CREATE TABLE RESERVA (

    id_reserva INT NOT NULL,

    cpf_usuario VARCHAR(11) NOT NULL,

    id_carona INT NOT NULL,

    status VARCHAR(20) NOT NULL,

    data_reserva DATE NOT NULL,

    CONSTRAINT PK_RESERVA PRIMARY KEY (id_reserva),

    CONSTRAINT FK_RESERVA_USUARIO FOREIGN KEY (cpf_usuario) REFERENCES
USUARIO(cpf),

```

```

        CONSTRAINT FK_RESERVA_CARONA FOREIGN KEY (id_carona) REFERENCES
CARONA(id_carona)
);

-- CRIANDO TABELA MENSAGEM

CREATE TABLE MENSAGEM (

    id_mensagem INT NOT NULL,

    cpf_remetente VARCHAR(11) NOT NULL,

    cpf_destinatario VARCHAR(11) NOT NULL,

    conteudo TEXT NOT NULL,

    data_envio DATE NOT NULL,

    CONSTRAINT PK_MENSAGEM PRIMARY KEY (id_mensagem),

    CONSTRAINT FK_MENSAGEM_REMETENTE FOREIGN KEY (cpf_remetente)
REFERENCES USUARIO(cpf),

    CONSTRAINT FK_MENSAGEM_DESTINATARIO FOREIGN KEY (cpf_destinatario)
REFERENCES USUARIO(cpf)
);

-- CRIANDO TABELA AVALIACAO

CREATE TABLE AVALIACAO (

    id_avaliacao INT NOT NULL,

    cpf_avaliador VARCHAR(11) NOT NULL,

    cpf_avaliado VARCHAR(11) NOT NULL,

    nota INT NOT NULL CHECK (nota BETWEEN 1 AND 5),

    comentario TEXT,

    data DATE NOT NULL,

    CONSTRAINT PK_AVALIACAO PRIMARY KEY (id_avaliacao),

```



```

        CONSTRAINT FK_AVALIACAO_AVALIADOR FOREIGN KEY (cpf_avaliador)
REFERENCES USUARIO(cpf),

        CONSTRAINT FK_AVALIACAO_AVALIADO FOREIGN KEY (cpf_avaliado)
REFERENCES USUARIO(cpf)

);

-- CRIANDO TABELA PAGAMENTO

CREATE TABLE PAGAMENTO (

    id_pagamento INT NOT NULL,

    id_reserva INT NOT NULL,

    valor NUMERIC(10,2) NOT NULL,

    status VARCHAR(20) NOT NULL,

    forma_pagamento VARCHAR(30) NOT NULL,

    data_pagamento DATE,

    CONSTRAINT PK_PAGAMENTO PRIMARY KEY (id_pagamento),

    CONSTRAINT FK_PAGAMENTO_RESERVA FOREIGN KEY (id_reserva) REFERENCES
RESERVA(id_reserva)

);

```

- **Inserção e atualização de dados (DML)**

Obs: Os comandos DML completos encontram-se no arquivo [unicarona_entrega3.sql](#), contendo mais de 120 registros por tabela conforme solicitado na atividade.

```

-- =====

-- PARTE 3: COMANDOS DE ATUALIZAÇÃO (UPDATES DO SISTEMA)

-- =====

UPDATE RESERVA SET status='Confirmada' WHERE id_reserva = 2;

UPDATE CARONA SET vagas_disponiveis = vagas_disponiveis - 1 WHERE
id_carona = 31 AND vagas_disponiveis > 0;

```

```
UPDATE USUARIO SET telefone = '85999998888' WHERE cpf = '0000000031';

UPDATE PAGAMENTO SET status='Aprovado', data_pagamento='2026-06-20'
WHERE id_reserva = 2;
```

5. Consultas SQL Desenvolvidas

```
-- CONSULTA 1: Relatório de Segurança - Monitoramento de CNHs Expirando

-- Esta consulta realiza uma junção interna (JOIN) entre as tabelas
MOTORISTA e USUARIO para projetar os dados cadastrais e de contato de
condutores específicos.

-- O filtro condicional (WHERE) atua restringindo o resultado a um intervalo
temporal de três anos, permitindo que a administração da plataforma
identifique

-- quais motoristas possuem habilitações vencidas ou com vencimento próximo
para fins de auditoria de segurança.

SELECT U.nome, U.telefone, M.numero_cnh, M.validade_cnh

FROM MOTORISTA M

JOIN USUARIO U ON M.cpf = U.cpf

WHERE M.validade_cnh BETWEEN '2026-01-01' AND '2028-12-31'

ORDER BY M.validade_cnh ASC;

-- CONSULTA 2: Ocupação Dinâmica de Viagens por Carona

-- Um relatório complexo que utiliza uma junção múltipla de seis tabelas
(Multi-JOIN) para consolidar todas as variáveis de uma viagem em uma única
linha

-- (motorista, veículo, pontos geográficos de origem e destino mapeados).
Além da projeção, a consulta executa uma expressão aritmética dinâmica no
SELECT,

-- subtraindo as vagas disponíveis da capacidade total do veículo para
retornar a quantidade exata de passageiros alocados por viagem.
```

```

SELECT C.id_carona, U.nome AS motorista, V.modelo AS veiculo,

       L1.nome_local AS origem, L2.nome_local AS destino,

       (V.capacidade_passageiros - C.vagas_disponiveis) AS
passageiros_alocados

FROM CARONA C

JOIN MOTORISTA M ON C.cpf_motorista = M.cpf

JOIN USUARIO U ON M.cpf = U.cpf

JOIN VEICULO V ON C.placa_veiculo = V.placa

JOIN LOCAL L1 ON C.id_origem = L1.id_local

JOIN LOCAL L2 ON C.id_destino = L2.id_local;

-- CONSULTA 3: Auditoria de Viagens com Faturamento Acima da Média Geral

-- Esta consulta implementa uma estrutura avançada de agregação e
subconsulta aninhada. O escopo interno agrupa as reservas e calcula a soma
dos pagamentos

-- aprovados por carona. A cláusula HAVING em tempo de execução atua
aplicando um pós-filtro que compara o faturamento de cada grupo individual
contra

-- a média aritmética global de repasses de todo o sistema, isolando apenas
as viagens de alta rentabilidade.

SELECT C.id_carona, C.data, C.horario, SUM(P.valor) AS faturamento_carona

FROM CARONA C

JOIN RESERVA R ON C.id_carona = R.id_carona

JOIN PAGAMENTO P ON R.id_reserva = P.id_reserva

WHERE P.status = 'Aprovado'

GROUP BY C.id_carona, C.data, C.horario

HAVING SUM(P.valor) > (

```

```

SELECT AVG(faturamento_total)

FROM (

    SELECT R2.id_carona, SUM(P2.valor) AS faturamento_total

    FROM RESERVA R2

    JOIN PAGAMENTO P2 ON R2.id_reserva = P2.id_reserva

    WHERE P2.status = 'Aprovado'

    GROUP BY R2.id_carona

) AS medias_gerais

);

```

-- CONSULTA 4: Identificação de Contas Inativas na Plataforma

-- Focada em métricas de engajamento, esta query utiliza junções externas à esquerda (LEFT JOIN) a partir da tabela USUARIO em direção às tabelas MOTORISTA

-- e RESERVA. Ao aplicar o filtro IS NULL nas chaves primárias dependentes, o banco de dados isola por negação lógica os alunos que criaram uma conta,

-- mas nunca interagiram no sistema como condutores ou passageiros.

```

SELECT U.cpf, U.nome, U.email

FROM USUARIO U

LEFT JOIN MOTORISTA M ON U.cpf = M.cpf

LEFT JOIN RESERVA R ON U.cpf = R.cpf_usuario

WHERE M.cpf IS NULL AND R.id_reserva IS NULL;

```

-- CONSULTA 5: Mapeamento de Perfis de Remetentes no Chat

-- Uma consulta de auditoria de comunicação que realiza a junção de caminhos (JOIN) entre a tabela de interações de texto (MENSAGEM) e a tabela de

```

-- cadastro básico para recuperar os nomes dos envolvidos no chat, ordenando
o fluxo de forma cronológica sequencial para análise de logs.

SELECT M.id_mensagem, U1.nome AS nome_remetente, U2.nome AS
nome_destinatario, M.conteudo, M.data_envio

FROM MENSAGEM M

JOIN USUARIO U1 ON M.cpf_remetente = U1.cpf

JOIN USUARIO U2 ON M.cpf_destinatario = U2.cpf;

-- CONSULTA 6: Rotas Universitárias de Alta Demanda

-- Esta consulta cruza a escala de viagens com a tabela LOCAL e a tabela
RESERVA. Os registros são agrupados de forma composta baseando-se nos nomes
textuais

-- de origem e destino. A restrição HAVING COUNT(*) filtra os agrupamentos
em tempo de execução, retornando apenas as rotas geográficas que possuem um

-- volume concentrado de requisições confirmadas (maior ou igual a 2).

SELECT L1.nome_local AS origem, L2.nome_local AS destino,
COUNT(R.id_reserva) AS total_reservas

FROM CARONA C

JOIN LOCAL L1 ON C.id_origem = L1.id_local

JOIN LOCAL L2 ON C.id_destino = L2.id_local

JOIN RESERVA R ON C.id_carona = R.id_carona

WHERE R.status = 'Confirmada'

GROUP BY L1.nome_local, L2.nome_local

HAVING COUNT(R.id_reserva) >= 2;

-- CONSULTA 7: Faturamento Líquido de Repasses por Condutor via Pix

-- Destinada ao fechamento financeiro, esta consulta realiza uma junção
quádrupla para rastrear o fluxo do dinheiro desde o PAGAMENTO, passando pela

```

-- RESERVA e CARONA, até chegar ao dono do veículo. A função de agregação SUM consolida os valores monetários, enquanto a cláusula WHERE filtra estritamente

-- transações com status liquidado ('Aprovado') e realizadas via 'PIX', agrupando o resultado final por motorista.

```
SELECT U.nome AS motorista, SUM(P.valor) AS receita_total_pix
FROM PAGAMENTO P
JOIN RESERVA R ON P.id_reserva = R.id_reserva
JOIN CARONA C ON R.id_carona = C.id_carona
JOIN MOTORISTA M ON C.cpf_motorista = M.cpf
JOIN USUARIO U ON M.cpf = U.cpf
WHERE P.status = 'Aprovado' AND P.forma_pagamento = 'PIX'
GROUP BY U.cpf, U.nome;
```

-- CONSULTA 8: Frota de Veículos Ociosos (Sem Registro de Viagens)

-- Esta consulta utiliza uma estratégia de otimização de subconsulta de exclusão por meio da cláusula NOT EXISTS. O banco mapeia todos os veículos cadastrados

-- na plataforma e remove do resultado aqueles cujas placas aparecem registradas em algum histórico de viagem na tabela CARONA, retornando a lista de carros ociosos.

```
SELECT V.placa, V.modelo, V.marca
FROM VEICULO V
WHERE NOT EXISTS (
    SELECT 1
    FROM CARONA C
    WHERE C.placa_veiculo = V.placa
);
```

```

-- CONSULTA 9: Controle de Qualidade - Ranking de Avaliação de Motoristas

-- Voltada à reputação dos condutores, a query extrai notas da tabela
AVALIACAO e faz um JOIN para identificar o motorista avaliado. É utilizada a
função estatística

-- AVG combinada com o operador de conversão e arredondamento matemático
ROUND para gerar uma nota média decimal limpa. O filtro HAVING restringe a
exibição

-- ao topo do ranking (notas de 4.0 a 5.0) e ordena os dados de forma
decrecente.

SELECT U.nome AS motorista, ROUND(AVG(A.nota)::numeric, 2) AS
media_reputacao

FROM AVALIACAO A

JOIN MOTORISTA M ON A.cpf_avaliado = M.cpf

JOIN USUARIO U ON M.cpf = U.cpf

GROUP BY U.cpf, U.nome

HAVING AVG(A.nota) >= 4

ORDER BY media_reputacao DESC;

-- CONSULTA 10 Relatório de Inadimplência e Pendências Financeiras

-- Relatório operacional de cruzamento multi tabelas que busca quebras de
conformidade no fluxo do sistema. Ela localiza estudantes que possuem
agendamentos de carona

-- ativos no sistema (status 'Pendente' ou 'Confirmada'), mas cujas
transações vinculadas na tabela de finanças constam com fluxo retido,
pendente ou cancelado,

-- exibindo os dados de contato do passageiro inadimplente.

SELECT U.nome AS passageiro, U.telefone, R.id_carona, P.valor, P.status AS
status_pagamento

```

```

FROM RESERVA R

JOIN USUARIO U ON R.cpf_usuario = U.cpf

JOIN PAGAMENTO P ON P.id_reserva = R.id_reserva

WHERE R.status = 'Pendente' AND P.status IN ('Pendente', 'Cancelado');

-- CONSULTA 11 Volumetria Mensal de Atividade Operacional (Universal)

-- Esta consulta utiliza funções escalares nativas de manipulação temporal
(EXTRACT) para decompor as datas das viagens em colunas isoladas de ano e
mês.

-- Os dados de volumetria são consolidados através da função contadora
COUNT(*) e agrupados de forma temporal, gerando um relatório histórico de
crescimento

-- de viagens da plataforma.

SELECT EXTRACT(YEAR FROM data) AS ano, EXTRACT(MONTH FROM data) AS mes,
COUNT(*) AS total_viagens

FROM CARONA

GROUP BY EXTRACT(YEAR FROM data), EXTRACT(MONTH FROM data)

ORDER BY ano DESC, mes DESC;

-- CONSULTA 12: Listagem Seletiva de Contatos de Passageiros por Carona

-- Uma query focada em fornecer listas de chamada limpas para o aplicativo.
Ela une a tabela de usuários com as tabelas de controle de vagas (RESERVA e
CARONA)

-- filtrando por status de assento validado ('Confirmada'). O resultado
projeta nome, e-mail e telefone de forma organizada e ordenada pelo
identificador da viagem.

SELECT C.id_carona, U.nome, U.email, U.telefone

FROM USUARIO U

```



```
JOIN RESERVA R ON U.cpf = R.cpf_usuario

JOIN CARONA C ON R.id_carona = C.id_carona

WHERE R.status = 'Confirmada'

ORDER BY C.id_carona, U.nome;
```

6. Anexos

- [unicarona_entrega3.sql](#)
- [UniCarona diagrama relacional.drawio](#)

7. Melhorias com relação à proposta da Entrega 2

Com base no *feedback* recebido na Entrega 2, foram realizadas algumas adequações no modelo de dados visando maior aderência aos conceitos de modelagem conceitual e relacional, bem como à implementação prática do banco de dados.

- **Adequação da representação de chaves:** Foram removidas as indicações de chaves estrangeiras diretamente nos atributos do Diagrama Entidade-Relacionamento (DER), mantendo essas referências apenas no modelo relacional, conforme as boas práticas de modelagem.
- **Reestruturação da especialização da entidade MOTORISTA:** A entidade MOTORISTA passou a ser modelada como uma especialização da entidade USUARIO. Dessa forma, o atributo *cpf* atua simultaneamente como chave primária e chave estrangeira, garantindo a herança dos dados cadastrais da entidade pai e evitando redundâncias.
- **Correção da modelagem das entidades MENSAGEM e AVALIACAO:** Foram adicionadas chaves estrangeiras referenciando a entidade USUARIO para identificar corretamente remetentes, destinatários, avaliadores e avaliados. Essa alteração garante maior integridade referencial e rastreabilidade das interações realizadas na plataforma.
- **Normalização dos dados de localização :** Foi criada a entidade LOCAL para armazenar informações geográficas utilizadas pelas caronas. Com essa alteração, os dados de origem e destino passaram a ser referenciados por meio de chaves estrangeiras, reduzindo redundâncias e promovendo maior padronização das informações armazenadas.